

SOLUCIÓN COMPLETA

Guía de Repaso N°2: Funciones en Python

Ejercicio 1 Temperatura corporal

★ Básico

10 pts

Puntaje: 10 pts

Solución modelo:

```
def clasificar_temperatura(temp):
    if temp < 36.0:
        return "Hipotermia"
    elif temp <= 37.5:
        return "Normal"
    elif temp <= 39.0:
        return "Fiebre"
    else:
        return "Fiebre alta"

temps = [35.5, 36.8, 38.1, 40.2]
for t in temps:
    print(f"{t} °C → {clasificar_temperatura(t)}")
```

Criterio	Pts	Escala
def con nombre y parámetro correcto	1 pt	1/0
if/elif/elif/else con los 4 rangos correctos	5 pts	5/4/3/2/1/0
return en cada rama	2 pts	2/1/0
Ciclo for con llamadas e impresión con f-string	2 pts	2/1/0



Aceptar if/elif encadenado o estructura con variables intermedias.
Los umbrales deben ser exactamente 36.0, 37.5, 39.0. Diferencia de umbral → -1pt por rango.

Ejercicio 2 Tabla de multiplicar

★ Básico

10 pts

Puntaje: 10 pts

Solución modelo:

```
def tabla(numero, limite=10):
    print(f'Tabla del {numero}:')
    for i in range(1, limite + 1):
        print(f'{numero} x {i:2} = {numero * i:3}')

tabla(6)
print()
tabla(4, limite=5)
```

Criterio	Pts	Escala
def con parámetro numero y default limite=10	2 pts	2/1/0
range(1, limite + 1) correcto (incluye límite)	3 pts	3/2/0
Impresión con formato dentro del for	3 pts	3/2/1/0
Llamada sin segundo argumento (default)	1 pt	1/0
Llamada con limite=5	1 pt	1/0



Error frecuente: range(limite) en vez de range(1, limite+1) → pierde los extremos. -2pts.
 Aceptar sin alineación de columnas (sin :2 y :3 en el f-string).

Ejercicio 3 Contar vocales

★ Básico

10 pts

Puntaje: 10 pts

Solución modelo:

```
def contar_vocales(texto):
    vocales = "aeiouAEIOU"
    contador = 0
    for c in texto:
        if c in vocales:
            contador += 1
    return contador

def porcentaje_vocales(texto):
    if len(texto) == 0:
        return 0.0
    n = contar_vocales(texto)
    return round(n / len(texto) * 100, 1)
```

```
frases = ["Hola Mundo", "Python es genial", "AEIOU"]
for f in frases:
    v = contar_vocales(f)
    pct = porcentaje_vocales(f)
    print(f'"{f}" → {v} vocales, {pct}%')
```

Criterio	Pts	Escala
contar_vocales: string de vocales con mayúsculas y minúsculas	2 pts	2/1/0
contar_vocales: for + if c in vocales + contador	3 pts	3/2/1/0
porcentaje_vocales: llama contar_vocales, divide, round(,1)	3 pts	3/2/1/0
Prueba con las 3 frases e impresión correcta	2 pts	2/1/0



Aceptar texto.lower() + vocales sin mayúsculas.
Verificar que porcentaje_vocales llame a contar_vocales y no reimplemente la lógica.

Ejercicio 4 Serie de Fibonacci

★★ Intermedio

10 pts

Puntaje: 10 pts

Solución modelo:

```
def fibonacci(n):
    a, b = 0, 1
    contador = 0
    while contador < n:
        print(a, end=' ')
        a, b = b, a + b
        contador += 1
    print()

print('fibonacci(8): ', end='')
fibonacci(8)
print('fibonacci(12): ', end='')
fibonacci(12)
```

Variante válida (acumular en lista):

```
def fibonacci(n):
    serie = []
    a, b = 0, 1
    while len(serie) < n:
        serie.append(a)
        a, b = b, a + b
```

```
print(' '.join(str(x) for x in serie))
```

Criterio	Pts	Escala
Inicializar a=0, b=1 antes del while	2 pts	2/1/0
Condición while contador < n (u equivalente)	2 pts	2/1/0
Actualizar a, b = b, a+b en cada iteración	4 pts	4/3/2/1/0
Incrementar contador y llamadas con 8 y 12	2 pts	2/1/0



La asignación simultánea a, b = b, a+b es la clave. Si usa variable temporal tmp también es válido.

Aceptar while len(lista) < n si acumula en lista.

Ejercicio 5 Número perfecto

★★ Intermedio

15 pts

Puntaje: 15 pts

Solución modelo:

```
def suma_divisores(n):
    suma = 0
    for i in range(1, n): # divisores propios: excluye n
        if n % i == 0:
            suma += i
    return suma

def es_perfecto(n):
    return suma_divisores(n) == n

# Buscar perfectos entre 1 y 1000
for num in range(1, 1001):
    if es_perfecto(num):
        print(f'{num} es perfecto')
```

Criterio	Pts	Escala
suma_divisores: range(1, n) excluye n correctamente	3 pts	3/2/0
suma_divisores: if n%i==0 y acumulador suma	3 pts	3/2/1/0
es_perfecto: llama suma_divisores y compara con n	4 pts	4/3/2/0
Ciclo range(1,1001) con if es_perfecto e impresión	3 pts	3/2/1/0
Encuentra correctamente 6, 28, 496	2 pts	2/1/0



Error frecuente: `range(1, n+1)` incluye al propio `n` → 6 suma $1+2+3+6=12$, no es perfecto. -3pts.
 Aceptar `range(1, n//2+1)` como optimización (los divisores propios no superan $n/2$).
`es_perfecto` debe llamar a `suma_divisores`, no reimplementar.

Ejercicio 6 Cifrado César

★★ Intermedio

15 pts

Puntaje: 15 pts

Solución modelo:

```
def cifrar(texto, desplazamiento):
    resultado = ''
    for c in texto:
        if c.isupper():
            nueva = chr((ord(c) - ord('A') + desplazamiento) % 26 +
ord('A'))
            resultado += nueva
        elif c.islower():
            nueva = chr((ord(c) - ord('a') + desplazamiento) % 26 +
ord('a'))
            resultado += nueva
        else:
            resultado += c # espacios, signos, etc.
    return resultado

def descifrar(texto, desplazamiento):
    return cifrar(texto, -desplazamiento)

original = "Hola Mundo"
cifrado = cifrar(original, 3)
descifrado = descifrar(cifrado, 3)
print(f"Original: {original}")
print(f"Cifrado: {cifrado}")
print(f"Descifrado: {descifrado}")
```

Criterio	Pts	Escala
cifrar: for c in texto, distingue mayúsculas/minúsculas	3 pts	3/2/1/0
cifrar: fórmula $(ord(c)-base+desp)\%26+base$ correcta	5 pts	5/4/3/2/1/0
cifrar: deja otros caracteres sin cambiar (else: resultado+=c)	2 pts	2/1/0
descifrar: llama cifrar con desplazamiento negativo	3 pts	3/2/0
Prueba con 'Hola Mundo' y resultado correcto	2 pts	2/0



La fórmula del módulo es la parte clave. Verificar que el % 26 permita rotar correctamente (Z→C con desp=3).

Aceptar que descifrar reimplemente la lógica con desp negativo en vez de llamar a cifrar.

Si solo maneja minúsculas: -3pts en criterio 2.

Ejercicio 7 Estadísticas de notas

★★★ Avanzado

15 pts

Puntaje: 15 pts

Solución modelo:

```
def promedio(notas):
    total = 0
    for n in notas:
        total += n
    return round(total / len(notas), 2)

def maximo(notas):
    mx = notas[0]
    for n in notas:
        if n > mx:
            mx = n
    return mx

def minimo(notas):
    mn = notas[0]
    for n in notas:
        if n < mn:
            mn = n
    return mn

def sobre_promedio(notas):
    prom = promedio(notas)
    return [n for n in notas if n > prom]

def reporte(notas):
    print("—— Reporte de Notas ——")
    print(f"Notas: {notas}")
    print(f"Promedio: {promedio(notas)}")
    print(f"Máxima: {maximo(notas)}")
    print(f"Mínima: {minimo(notas)}")
    print(f"Sobre prom: {sobre_promedio(notas)}")

datos = [45, 72, 88, 55, 91, 63, 77, 49, 84, 60]
reporte(datos)
```

Criterio	Pts	Escala
promedio: acumulador for sin sum(), round(,2)	3 pts	3/2/1/0
maximo: inicializa notas[0], for+if sin max()	2 pts	2/1/0
minimo: misma lógica sin min()	2 pts	2/1/0

sobre_promedio: llama promedio() y filtra la lista	4 pts	4/3/2/1/0
reporte: llama las 4 funciones e imprime correctamente	4 pts	4/3/2/1/0



Si usa sum(), max() o min(): -2pts por función afectada.
 sobre_promedio puede usar list comprehension o for + append, ambas válidas.
 Verificar que reporte() llame a cada función y no reimplemente la lógica.

Ejercicio 8 Validar RUT

☆☆☆ Avanzado

15 pts

Puntaje: 15 pts

Solución modelo:

```
def limpiar_rut(rut_str):
    return rut_str.replace('.', '').replace('-', '')

def calcular_dv(numero):
    suma = 0
    multiplicador = 2
    while numero > 0:
        suma += (numero % 10) * multiplicador
        numero //= 10
        multiplicador += 1
        if multiplicador > 7:
            multiplicador = 2
    resto = 11 - (suma % 11)
    if resto == 11:
        return '0'
    elif resto == 10:
        return 'K'
    else:
        return str(resto)

def validar_rut(rut_str):
    limpio = limpiar_rut(rut_str)
    numero = int(limpio[:-1]) # todo menos el último carácter
    dv_ingresado = limpio[-1].upper()
    dv_calculado = calcular_dv(numero)
    return dv_ingresado == dv_calculado

ruts = ["12.345.678-9", "11.111.111-1", "7.654.321-K"]
for rut in ruts:
    estado = "Válido" if validar_rut(rut) else "Inválido"
    print(f"{rut} → {estado}")
```

Criterio	Pts	Escala
limpiar_rut: reemplaza puntos y guión correctamente	2 pts	2/1/0

calcular_dv: while para extraer dígitos (numero%10 y !=10)	4 pts	4/3/2/1/0
calcular_dv: secuencia multiplicadores 2-7 que se reinicia	3 pts	3/2/1/0
calcular_dv: casos resto==11→'0', resto==10→'K', else str()	3 pts	3/2/1/0
validar_rut: llama limpiar, extrae número y DV, compara	3 pts	3/2/1/0



Este es el ejercicio más complejo. Aceptar implementación con lista invertida de dígitos en vez de while.

Si extrae los dígitos con `str(numero)[::-1]`: válido.

El DV '0' (no cero el número) es un caso especial que indica un RUT con DV=11 que se convierte en 0.